

Phase 4 — 系统统计监控

SPEC-010 | Status: 设计中 | 依赖: SPEC-004 (Scheduler + Redis) , SPEC-008 (Skill 产生可统计的 Agent/Task 数据)

目标

前置依赖检查

前置 Spec	状态	备注
SPEC-002	✅/❌	CI Pipeline 就绪
SPEC-003	✅/❌	Redis 连接层可用
SPEC-004	✅/❌	Scheduler + Agent Service 可用
SPEC-008	✅/❌	Skill 实现就绪 (产生可统计的 Agent/Task 数据)

实现系统级统计监控 (Redis Stats Counter)、Dashboard ROI 投入产出比计算、MongoDB TTL 日志自动清理。

背景

Roadmap Phase 4 Week 8 附加, P4-25 ~ P4-26, P4-13 (TTL 扩展), 总计 ~8h。

详细设计

1. Redis Stats 计数器

- Scheduler 定时任务 (每 5 分钟) 直接写入 Redis Pipeline (不经过消息队列, 仅记录日志)
- Redis 必须开启 AOF + RDB 持久化
- 统计指标:

指标	Redis Key	说明
Agent 调用次数	<code>stats:agent_calls:{date}</code>	按日聚合，区分 sync/async
模型调用次数	<code>stats:model_calls:{date}:{model_name}</code>	按模型+日期聚合
Session 创建次数	<code>stats:sessions:{date}</code>	按日聚合
Task 创建次数	<code>stats:tasks:{date}</code>	按日聚合
Token 消耗量	<code>stats:tokens:{date}:{model_name}</code>	input/output 分计
AI 总成本	<code>stats:cost:{date}</code>	按日聚合，单位分

2. Dashboard ROI 计算

ROI = 等效节省人时 / AI 总成本 (元)

等效节省人时 = Agent 调用次数 × 30min / 60

AI 总成本 = sum(stats:cost:{date}) / 100

Dashboard API: GET /admin/dashboard/roi

3. MongoDB TTL 自动清理

Collection	TTL	说明
<code>audit_logs</code>	90 天	审计日志
<code>request_logs</code>	30 天	请求日志
<code>sessions</code>	30 天	已过期会话
<code>notifications</code>	7 天	通知记录
<code>token_consumptions</code>	90 天	Token 消耗（长期聚合从 Redis Stats 取）

MongoDB TTL 索引 `{created_at: 1, expireAfterSeconds: N}`，后台每 60s 扫描一次自动删除，无需手动 Worker。

可行性分析

检查项	结论
是否需要新 DB 集合	Yes (output_stats, roi_metrics)
是否影响现有 API	Yes (新增 Dashboard stats/roi API)
性能影响	Redis Pipeline 写入 < 5ms, 5min 窗口丢失可接受
是否需要新增 Skill	No
是否需要 E2E 测试	Dashboard ROI/KPI 图表渲染 UI 用例

相关文件

文件	角色	变更幅度
<code>internal/logic/stats_collector.go</code>	Redis Stats 收集器	新建
<code>internal/service/admin/stats.go</code>	Dashboard Stats API	新建

验证标准

1. Scheduler 每 5 分钟写入 Redis, 计数器正确递增
2. Redis 重启后 AOF+RDB 恢复, 统计数据不丢失
3. Dashboard ROI 数值与 Token 成本计算一致
4. 审计日志 90 天后自动过期 (可验证 TTL index 存在)